Guía Teórico-Práctica: Taller de Python Avanzado

Construyendo tu Tamagotchi-Bot desde cero

IEEE RAS Student Branch Chapter
Universidad de Concepción

18 de Marzo, 2026

Índice

1. Introducción a Python: Fundamentos y Gestión de Memoria **2**
1.1. Comunicándonos con la consola (`print` e `input`) 2
1.2. Variables y Tipos de Datos (Guardando info en la RAM) 2
1.3. F-Strings (Imprimiendo variables de forma bonita) 3

2. Lógica y Toma de Decisiones **3**
2.1. Operadores Relacionales y Sintaxis Lógica (`and/or`) 3
2.2. El Flujo de Ejecución: `if`, `elif`, `else` 3

3. Guardando mucha memoria junta: Las Listas **4**
3.1. Listas y el misterio del Índice Cero 4
3.2. Modificando y Contando 4

4. Ciclos: Haciendo que el código se repita solo **4**
4.1. El ciclo `for` y el `while` 4

5. Conceptos Pro (Bases para el Mega Desafío) **5**
5.1. Revisando listas automáticamente y mutándolas (`.append / .remove`) 5
5.2. Lógica Anidada (Los Ifs dentro de Ifs) 5

6. Mega Desafío Final: Construye tu Tamagotchi desde Cero **6**
6.1. Misión Bronce: El Despertar del Núcleo Básico del Robot 6
6.2. Misión Plata: Límites Físicos y Almacenamiento 6
6.3. Misión Oro: El Easter Egg Desbloqueable IEEE 6

1. Introducción a Python: Fundamentos y Memoria

Python es un lenguaje bacán para empezar porque está diseñado para ser súper legible. Básicamente, Python hace toda la "pega sucia"por debajo, como administrar la memoria del compu, para que nosotros nos preocupemos solo de la lógica. En este taller vamos a usar **Google Colab**, que es una herramienta genial porque nos permite programar directo en el navegador de internet sin tener que calentarnos la cabeza instalando programas raros.

1.1. Hablando con la consola (`print` e `input`)

Cualquier programa decente necesita interactuar con nosotros. Para hablar, usamos la función `print()`, que tira mensajes a la pantalla. Si le pasamos varias palabras separadas por comas, Python es buena onda y le pone un espacio entre medio por su cuenta.

Por el otro lado, para escuchar, ocupamos `input()`. Ojo acá: cuando tu código llega a un `input()`, **el programa entero se congela**. Se queda ahí esperando hasta que escribas algo con el teclado y apretes la tecla Enter para soltarlo.

```python
# Tirando varias palabras separadas por comas a la pantalla:
print("Estado del robot:", "Optimo", "-", "Sensores online")

comando = input("Escribe tu primer comando: ")
```

1.2. Variables y Tipos de Datos (Guardando info en la RAM)

Hagamos cuenta que la memoria RAM de tu equipo es un gran estante lleno de cajas. Cuando creas una variable (por ej. `bateria = 100`), Python pesca una caja vacía, le mete el número 100 adentro, y le pega un post-it que dice "bateria"para que podamos encontrar esa caja más rato.

Como el compu es súper cuadrado, hay **reglas de oro** al momento de bautizar estas cajas:

1. **Cero espacios:** El computador se marea con los espacios. En vez de eso, usamos un guión bajo para conectar las palabras (el clásico `snake_case`, ej. `vida_maxima`).

2. **No parten con números:** `robot1` es un nombre válido, pero `1robot` te va a tirar un error fatal altiro.

3. **Ojo con las mayúsculas:** Python diferencia entre mayúsculas y minúsculas. `Bateria` y `bateria` van a ser dos cajas totalmente distintas y nada que ver entre sí.

Aparte, el computador necesita saber *qué tipo* de dato está guardando, porque lógica pura: no puedes sumar la palabra "Holaçon el número 5.

- **Strings (`str`):** Puro texto y palabras. Siempre, pero siempre, tienen que ir abrazados por comillas .

- **Enteros (`int`):** Números puros para hacer matemáticas. Van sin comillas.

- **Booleanos (`bool`):** Es como un interruptor. Solo pueden ser Verdadero (`True`) o Falso (`False`). Fíjense que parten con mayúscula.

- Alerta importante: Todo lo que el usuario escriba a causa de un `input()` entra a la consola siendo catalogado como un Texto (String). Si le pides un número y quieres sumarlo, **tienes que** convertir este texto a número a lo bruto, usando la función `int()`.

```python
# Creando nuestras primeras cajas en la memoria:
nombre_robot = "Wall-E" # Tipo: Texto (String)
bateria = 100           # Tipo: Numero Entero (Integer)
escudos_activos = True  # Tipo: Booleano
```

```python

# Pidiendo un numero (entra como texto) y convirtiendolo a int en la misma linea:
energia = int(input("¿Cuanta energia extra quieres inyectar? "))
```

1.3. F-Strings (Imprimiendo variables de lujo)

Las "F-strings.ªpañan muchísimo en Python. Si le pones una letra `f` justo afuerita de las comillas de tu texto, puedes meter tus variables allá adentro usando llaves {}. Lo mejor de todo es que puedes hacer matemáticas ahí adentro y Python lo va a calcular antes de disparar el texto final.

```python
vida_base = 100
# El programa saca la cuenta de 100 + tu energia justo en el momento de imprimir
print(f"Hola {nombre_robot}, tu bateria final ascedera a {vida_base + energia}%.")
```

2. Lógica y Toma de Decisiones

2.1. Comparando cosas (`and`/`or`)

Acá entramos a la lógica dura. Cuando ocupamos símbolos matemáticos como mayor que (`>`), menor que (`<`), o doble igual (`==`), Python evalúa tu pregunta matemática y te responde al seco con un valor booleano: `True` si lo que le preguntaste es verdad, o `False` si es mentira. Anotense esto porque siempre hay errores: usar un puro signo igual (`=`) significa **"guarda este dato en memoria"**, mientras que el doble signo igual (`==`) significa **çompárame a ver si esto es igual a esto otro"**.

Y podemos mezclar varias preguntas juntándolas con los **operadores lógicos**:

- `and` ("Y"): Es súper estricto. Te exige que **todas** las condiciones se cumplan al mismo tiempo para decir que es `True`.
- `or` (.º"): Es relajado. Le basta con que al menos **una sola** cosa sea cierta para tirar `True`.

```python
peligro = True
escudos = 0

# Como ya no te quedan escudos y ademas estai en peligro... esto tira True
print(escudos == 0 and peligro == True)
```

2.2. El Camino del Tren: `if`, `elif`, `else`

Normalmente Python lee tu código de arriba a abajo, derechito. Las palabras `if`, `elif` y `else` vienen siendo como el cambio de vías de un tren, eligiendo por qué ruta de código se va a ir la ejecución dependiendo de si las condiciones dieron verdaderas o falsas. Es importante cachar que solo se ejecuta UN bloque. Si el primer `if` le achuntó y era verdad, se mete ahí y patea todo lo demás para abajo.

Acá entra uno de los temas más cuáticos de Python: la **Indentación** (los espacios al inicio de la línea). Pasa que en otros lenguajes usan hartas llavecitas o corchetes para indicar qué código va adentro de un `if`, pero a Python le gusta lo visual. Así que apretar Tab (o dar 4 espacios) es sagrado y **obligatorio** para que el intérprete sepa que ese trozo de código está .ªdentro"de tu `if`.

```python
ataque_enemigo = 150
modo_defensa = True

# Simulando una batalla evaluando dos cosas a la misma vez:
if ataque_enemigo > 100 and modo_defensa == False:
    print("¡Impacto critico! Danos severos detectados.")
elif ataque_enemigo > 100 and modo_defensa == True:
    print("Bloqueo del escudo antimateria exitoso. Energia conservada al limite.")
else:
    # El else te salva atrapando cualquier otra cosa que te falto calcular.
    # Por ejemplo, si el ataque enemigo de pura casualidad cayera a 50 puntos.
    print("Ataque debil repelido.")
```

3. Agrupando info: Las benditas Listas

3.1. Listas y por qué contamos desde Cero

En un proyecto real vais a necesitar juntar mucha info junta de golpe. Una `lista` reserva un saco enorme de memoria donde poder ir metiendo todo tipo de variables sueltas adentro de unos corchetes `[]`. En Python son una maravilla, puedes hacer un cóctel de números, textos y booleanos en un solo grupo sin que colapse el programa.

¿Y por qué chuta partimos contando desde el 0?: Para saber en qué cajita está nuestra info dentro de la lista, usamos un "Índice". Resulta que desde los inicios de la computación, para los procesadores era harto más fácil medir la "distancia.º en qué lugar quedabas partiendo del inicio de la lista. Onda, el primer elemento obviamente queda a una distancia de cero pasos (en el **índice 0**), el segundo compañero ya se corrió a una distancia de 1 paso (índice 1), y así para adelante. Resumen: todos los computadores parten contando desde el cero. Acostúmbrese.

```python
# Una mochila guardando puros textos
mochila = ["Tuerca", "Bateria", "Circuito"]

# Un sensor que agrupa datos crudos como enteros
lecturas_cm = [15, 12, 10, 8, 2]

# Una lista totalmente pelada y vacia (para despues ir echandole cosas)
inventario_vacio = []
```

3.2. Modificando y Contando (`len`)

Cambiar un ítem de tu lista es re fácil. Lo ubicas usando su índice matemático y le tiras un `=` encima. Sería equivalente a sacar algo del cajón y meter algo distinto al fondo. Si de repente en medio del código te pierdes y necesitas cachar cuántos elementos hay acumulados en tu lista, usas la función regalona `len()` (de largo, ah).

```python
cantidad = len(mochila) # Si abrimos la mochila y contamos, nos va a arrojar un "3"

# Borramos a la mala la vieja "Bateria" y justo en su posicion [1] dejamos el Laser
mochila[1] = "Laser Usado"
```

4. Ciclos: Haciendo la pega por nosotros

4.1. El ciclo `for` y el `while`

Es una lata tremenda copiar y pegar el mismo bloque de código 50 veces seguidas. Los Çiclos"(o loops) la hacen corta, repitiendo el bloque de código que les pongas en su guata de

maneral brutal.

El **ciclo** `for` es bacán cuando tienes tus cosas guardadas en una lista y quieres automatizar. Su pega es recorrer ítem por ítem, hacer algo con lo que agarró, y pasarse derechito al ítem de al lado. Se detiene sólito apenas la lista se termine.

Por otro lao', el **ciclo** `while` es el rey indiscutible de los videojuegos. Va a repetir tus instrucciones a toda máquina sin parar **mientras** su condición inicial le siga diciendo que es `True`.

¡Mucho ojo con el Loop Infinito!: ¡Chicos, si abren un 'while', ustedes mismos tienen la responsabilidad de asegurar que en algún minuto se frene! Pónganle que arman un 'while' que revisa `bateria = 100`, asegúrense que esa batería empiece a gastarse en algún punto del código (`bateria = bateria - 1`). Si uno se pajaronea y olvida hacer que esa condición llegue a `False`, su programa va a quedar rodando eternamente a máxima velocidad hasta comerse la compu entera por un loop infinito. Obliga a cerrar el terminal a la mala.

Pa' salvarnos el pellejo en casos extremos como ese, existen los frenos de mano. La palabra `break` te hace bolsa el ciclo en seco y te expulsa echando humos hacia la salida, pa' seguir con el código principal. La compañera de esa es `continue`, que te dice: .ºk, chao lo que quede abajo, me salto el resto y vuelvo a comenzar la vuelta del ciclo desde arriba altiro".

```python
# FOR: Simplemente recorriendo la lista haciendo una clasica Cuenta Regresiva
conteo = [3, 2, 1]
for numero in conteo:
    print(f"Lanzamiento en... {numero}!")
print("¡DESPEGUE!")


# WHILE: Pidiendole al usuario duro y parejo la contraseña correcta
acceso = False
while acceso == False:
    # Si nunca llegas a mutar el boolean 'acceso', quedai atrapado pa siempre!
    clave = input("Ingresa protocolo de desencriptacion: ")

    if clave == "1234":
        print("Certificados Validos Encontrados.")
        # La solucion normal y esperada: cambiar la variable pal While cierre por fuera
        acceso = True

    if clave == "ABORT":
        print("Autenticacion cancelada.")
        # La solucion a lo bruto: Destruir el While a la fuerza
        break
```

5. Trucos Pro para el Desafío

5.1. Buscando y sacando cositas de las Listas (`.append` / `.remove`)

A ver, si necesitas cachar al vuelo si tu bot ya agarró la "llavez la tiene en el inventario, es latero revisar la lista entera uno por uno usando un `for`. Paren el leseo, mejor ocupen la la frase mágica `in`. Python hace la pega por ustedes y revisa todo como un rayo diciéndoles si está ahí o no.

Las listas crecen o se achican constantemente. Usen `mochila.append(elemento)` y con eso agregarán un chuncho nuevo directo al fondo de su inventario. Con `mochila.remove(elemento)` le mandan a Python a que busque esa palabra en particular y me la borre de ahí. Pero pónganse víos: eso solo elimina la primera ocurrencia que pille, y lo peor: te tira tremendo pantallazo rojo de error si es que le pides que elimine algo que nunca estuvo en la mochila pa' empezar. Sean cautos.

```python
inventario = ["Agua", "Comida", "Agua", "Comida", "Roto"]

# Ocupando la palabra 'in' para buscar cosas velozmente sin calentarse la cabeza
if "Agua" in inventario:
    print("Consumiendo insumo hidrico...")
    inventario.remove("Agua")

# Cacha como desaparecio UNA pura botella de la lista
# >> ["Comida", "Agua", "Comida", "Roto"]
```

5.2. El If adentro de otro If (Lógica Anidada)

La pulenta es que tú puedes poner `if` adentro de las guatas de otros `if`. Cada vez que embutas uno adentro, tienes que respetar la sangría y empujarlo un Tabulazo más a la derecha. Pasa en todos los juegos, es un clásico: "Ya mira, compadre, te tomaste una poción, te sumo 20 puntos piola. Pero espera un ratito, metamos un if más dentro para chequear: si por sumarte eso, superaste el límite de los 100 de vida intocable, pesco los niveles y te los dejo clavados en esos 100 pelados para que no pitees las reglas físicas del juego". Lógica pura.

```python
if op == "Cargar":
    energia = energia + 20

    # Este if de aca es el parche curita pa que los niveles no rebalsen de 100
    if energia > 100:
        energia = 100
```

6. Mega Desafío Final: Armando desde la nada tu Tamagotchi

Llegó la hora de la verdad. Nada de copiar código ni arreglar cosas fomes; ahora vas a armarle el cerebro a un Tamagotchi-Bot interactivo partiendo con una **hoja en blanco**. Ve haciéndolas por parte y verás cómo vas armando el motor de videojuegos. ¡Que no decaiga!

6.1. Misión Bronce: Despertando el Núcleo del Robot

Primero armemos su centro neurálgico, su corazón. Ojo a estos pasos:

1. Pídele al jugador que le ponga un nombre al bot con su buen `input()` y guárdalo en una variable. Modela el cuerpecito creando el medidor: `energia = 100` y el interruptor vital lógico `encendido = True`.

2. Échale chispa y dale motor armando el inicio del juego con el famosillo bucle infinito: mete un tremendo `while encendido:`

3. De entradita adentro de este Bucle, sácate una F-String chora para la pantalla mostrando stats, y ponte un menú de toda la vida pidiendo elegir: `[1] Jugar`, `[2] Cargar`, o `[3] Apagar`. Y obvio, salva el resultado en un inputazo.

4. Tírale las ramas con el `if / elif / else` clásico. Si aprietan Çargar", métete a meterle 25 a su energía. Si el socio le da a "Jugar", se gasta, quítale unos 20 de energía. Si se achacan y marcan .Apagar", bájale las cortinas cambiando `encendido = False` o, si andai bélico, pégale flor de patada con un `break` pa' que lo saque del bucle de golpe y porrazo.

6.2. Misión Plata: Topes Físicos y la Bendita Mochila

Cabros, nos llamaron los de IEEE, resulta que el juego está rotísimo: los giles cargan el Tamagotchi al infinito y rompen la matrix de energía:

1. **Límites Lógicos Anidados:** Métete de cabeza a los bloques `if` de cargar y jugar que codeaste recién, y encájales unos `if` extra más chiquititos, como filtros (apréndanse la indentación, córranlos pa' la derecha). El objetivo es que a la mala frenen que la energía no nos pase los 100 pa' arriba ni rebote bajo el 0, matándonos a negativos.

2. **La Mochila de Invientario:** Ármate una lista vacía super piola, onda `mochila = []`. **CUIDADO CON ESTO CAUROS:** Péguenla sí o sí **afuerísima** y siempre antes de tu ciclo gigante de `while`; si son pajaronazos y la ponen adentro del while, el compu te la va a vaciar en cada jugada perdiendo toda la plata.

3. Modifiquen el menú, háganle una patita que sea la gran `[4] Explorar`. En esa rama háganle la pregunta a lo Dora The Explorer: "¿Qué cachivache raro pillaste por la orilla?". Luego pescan la respuesta y se la zampan enterita pa' adentro de la mochila apañando con la función `mochila.append()`.

6.3. Misión Oro: El súper Easter Egg IEEE de la Directiva

Nos pusimos maldosos y queremos meter un código secreto para hackear el sistemita:

1. Ponle entremedio y medio escondido un `elif` al menú maestro, que se dedique en alma a esperar si el mate'wea pone textualmente y sin titubear las siglas `ÏEEE"` solitas.

2. En la chicharra de ese bloque tírate el mensaje colosal: **"¡OH LALÁ! MODO DIOS DEVELOPER DESBLOQUEADO!"**. Y acto seguido pescan la pobre variable d' `energia` y pónganle a la maletería nomás `9999` puntos pal perla, regalados. Aparte de tirar un `append` a la mochila metiendo "Motor Cuántico de UdeC"de reliquia o algo brígido que se les ocurra. ¡Ya estariamos, tienen el juego completado!.